Ansatz: Cost-Model Routing for Verified Electromagnetic Simulation in Superconducting Qubit Design

Ansatz AI
contact@useansatz.ai

August 2026 (v0 technical report)

Abstract

The inner loop of superconducting qubit design is electromagnetic simulation: each geometry iterate requires a multi-conductor capacitance extraction whose lumped-oscillator reduction yields the Hamiltonian parameters designers actually target. We describe the v0 Ansatz engine, which accelerates this loop with two coupled components. First, a forward surrogate trained on 1,934 experimentally validated Ansys Q3D capacitance matrices (the SQuADDS database) predicts capacitance matrices at microsecond latency, roughly 5× more accurately than nearest-design lookup in distribution (0.31% vs. 1.75% mean absolute percentage error), with downstream coupling and dispersive-shift errors of 0.70% and 1.71%. Second, because every data-driven predictor degrades outside its training hull, a *routed, verified* field-solver answers arbitrary designs: a cost-model router chooses per instance among sparse-direct, AMG-preconditioned Krylov, surrogate-initialized matrix-free Krylov, and hybrid pipelines, every pipeline terminates in residual verification, and a runtime escalation monitor bounds the cost of routing mistakes. On geometry distributions anchored to the same validated design family, the routed solver tracks the per-instance oracle within measurement noise and dominates every fixed pipeline choice as resolution varies, while HINTS-style fixed hybrid schedules are never competitive. We argue this instance-level, verification-first reformulation of learned solver routing — rather than per-iteration classification or uncertainty-quantified surrogation alone — is the form in which neural acceleration becomes deployable in engineering design loops.

1 Introduction

Superconducting quantum processors are designed in a loop whose inner step is electromagnetic simulation. A designer adjusts planar geometry — arm lengths of a cross-shaped transmon island, the reach of a readout claw, etch gaps to the ground plane — and asks what Hamiltonian the physical chip will realize: charging energy E_C, qubit frequency f_q, anharmonicity α, readout coupling g, dispersive shift χ. The workhorse answer is a multi-conductor capacitance extraction (Ansys Q3D in most labs) followed by a lumped-oscillator-model (LOM) reduction [4, 6]. Each extraction costs minutes; design loops, frequency-allocation studies, and yield analyses run hundreds to thousands of them. This loop, not the final sign-off simulation, is where design time actually goes.

Neural surrogates promise millisecond predictions, and in distribution they deliver (Section 4). The deployment problem is what happens *off* distribution: a surrogate extrapolating outside its training hull fails silently, and in a domain where a misallocated qubit frequency means a dead chip after a multi-week fabrication cycle, silent failure is disqualifying. The established alternatives are unsatisfying in the other direction: fixed hybrid schedules such as HINTS [2] interleave a neural

corrector with classical relaxation on a rigid cadence, paying surrogate costs whether or not they help; per-iteration learned routing over solver ensembles [3] adapts more finely but optimizes iteration progress rather than the quantity a practitioner buys hardware for: wall-clock to a *verified* answer.

This report describes the v0 Ansatz engine, built around two commitments. First, **verification is unconditional**: every field the engine returns satisfies a relative-residual tolerance on the target discretization, checked directly, with a sparse-direct fallback on failure. Learned components can only spend or save time; they cannot corrupt results. Second, **routing is an economic decision made per instance**: a cost model predicts the wall-clock of each available pipeline for the design, grid, and tolerance at hand, the cheapest is chosen, and a runtime monitor escalates — once, warm-started — if the observed convergence slope betrays the prediction. The result is an engine whose worst case is the classical solver plus milliseconds, and whose common case is substantially faster.

2 Setting

Device family. We target the xmon-style transmon with readout claw: a cross-shaped island in an etched pocket of a grounded plane, one arm wrapped by a C-shaped coupler. This family is the backbone of the SQuADDS validated design database [1], which records, for 1,934 designs, the geometry and the Ansys Q3D capacitance matrix (adaptively refined to 0.1%). The database varies three parameters (cross length 90–420 μm, claw length 70–400 μm, ground spacing 4.1–10 μm); our generated benchmark family varies all seven (widths, gaps included) around the same anchors.

From fields to Hamiltonians. With conductors $\{1, \ldots, K\}$ embedded as Dirichlet regions, excitation k solves the Laplace problem with conductor k at unit potential; discrete Gauss-law charges assemble the Maxwell capacitance matrix C. The LOM reduction maps $C_q = |C_{\text{cross,gnd}}| + |C_{\text{cross,claw}}|$ and $C_c = |C_{\text{cross,claw}}|$ through exact Cooper-pair-box diagonalization to (f_q, α, E_C) and through the exact lumped coupling formula to g, with the beyond-RWA second-order dispersive shift χ [4, 1]. Designers target the Hamiltonian quantities; the capacitance matrix is the interface our engine must produce quickly and credibly.

Discretization. v0 solves 2D plan-view electrostatics on uniform grids ($n \in \{255, 511, 1023, 2047\}$, embedded Dirichlet masks, matrix-free 5-point operators alongside assembled CSR forms). Verified accuracy is defined against this discretization; the realism of the family rests on the geometry anchoring above and on Tier 1 being trained on real Q3D matrices. The 3D solver backend (AWS Palace) is roadmap, not v0.

3 Method

3.1 Tier 1: forward surrogate on validated Q3D data

Tier 1 is deliberately unglamorous: per-entry gradient-boosted regressors mapping the varied geometry parameters to the five distinct capacitance-matrix entries, trained on the 1,934 SQuADDS Q3D extractions. Inference is microseconds on a CPU; retraining is seconds. We benchmark against the methods a practitioner would otherwise use on the same database: nearest-design lookup (the local behavior of database interpolation), distance-weighted kNN, a global linear fit, and a random forest. SQuADDS's own published interpolation is a physics-based scaling law around the nearest design [1]; no forward ML surrogate ships with the database, which is precisely the gap Tier 1 fills.

Because designers consume Hamiltonian parameters rather than capacitances, we report both the capacitance error and its LOM image (f_q, α, g, χ).

3.2 Tier 2: routed, verified field solves

Tier 2 answers the designs Tier 1 cannot be trusted on. The unit of work is a *design solve*: K excitations sharing one geometry, hence one sparsity structure — so assembly, factorization, AMG hierarchy, and surrogate batch inference are computed once per design and shared, exactly as a practitioner would amortize them. The pipelines are:

- **direct**: assemble once; sparse LU; K triangular solves.
- **amg_cg**: assemble once; smoothed-aggregation AMG setup (pyamg [5]); CG per excitation, hierarchy reused.
- **surr_cg**: U-Net field prediction per excitation (batched, one GPU pass); matrix-free CG from the predicted iterate; *no assembly, no setup.*
- **surr_amg**: surrogate init, then AMG-CG polish (buys AMG's convergence when the prediction is good but CG stalls).
- **hints**: the fixed 1-in-16 corrector schedule of [2] with the same U-Net as corrector — the fixed-schedule control.

Cost-model router. Ten features describe an instance: the seven geometry parameters, $\log_2 n$, free-node fraction, and $-\log_{10}\tau$. Per-pipeline gradient-boosted regressors predict log wall-clock; the router takes the argmin. Decisions cost microseconds, the model is a few hundred kilobytes, and refitting on a customer's own instance mix is a seconds-long job on measurements the engine logs anyway. We chose this over per-iteration classification [3] deliberately: the object with economic meaning is the total verified solve time, the per-design pipeline space is small and structured, and instance-level regression makes the router's errors interpretable (a mispredicted cost, not an opaque policy).

Escalation monitor. Iterative pipelines log $(t, \log_{10}\|r\|)$ per block; a linear projection of the recent slope estimates the finish time. If the projection exceeds the predicted cost of the best alternative (plus the elapsed segment), the solve escalates to that alternative exactly once, warm-started from the current iterate. This converts cost-model errors from unbounded stalls into a bounded, observable event.

Verification. Every pipeline ends by evaluating the true relative residual on the target grid. If it exceeds τ (never observed in our benchmarks), the engine silently reruns the direct pipeline. The guarantee to the user is therefore discretization-exact accuracy at τ, independent of every learned component.

3.3 Guardrail properties

Proposition 1 (Verification safety)**.** *Every field returned by any pipeline satisfies* $\|b - Au\|_2 / \|b\|_2 \le \tau$ *on the target discretization; on verification failure the engine falls back to the sparse direct solve, so the routed result equals the classical result in the worst case.*

model	capacitance MAPE iid	capacitance MAPE out-of-hull	downstream (iid) g	downstream (iid) χ
nearest-design lookup	1.75%	3.50%	4.03%	28.7%
kNN-5 (distance-weighted)	1.11%	3.96%	2.17%	13.6%
linear	3.02%	3.50%	7.79%	20.7%
random forest	2.19%	4.12%	5.01%	15.5%
Ansatz (GBR)	**0.31%**	3.51%	**0.70%**	**1.71%**

Table 1: Forward model vs. practitioner alternatives on held-out real Ansys Q3D capacitance matrices (SQuADDS). Downstream errors propagate predictions through the LOM at $L_j = 10\,\mathrm{nH}$, $f_r = 6.116\,\mathrm{GHz}$.

Proposition 2 (Bounded escalation regret)**.** *Let t_s be the time spent in a monitored pipeline before escalation fires, and T_{alt} the (warm-started) time of the escalation target. The routed cost is at most $t_s + T_{\mathrm{alt}}$, where t_s is bounded by the monitor's projection rule; in particular a stalled pipeline cannot consume more than the patience window beyond its projected-competitive segment.*

4 Experiments

All experiments run on a single Apple M4 Pro (24 GB); every number is reproducible from fixed seeds via `scripts/run_all_benchmarks.sh` in the open-source repository. The verified tolerance is $\tau = 10^{-8}$ (relative residual) throughout.

4.1 Forward model on real Q3D data

Table 1 scores Tier 1 on the 1,934 SQuADDS Q3D extractions under (a) a random 85/15 split and (b) an out-of-hull split holding out the largest 12% of `cross_length`. In distribution, the gradient-boosted model reaches 0.31% mean capacitance error — 5.6× better than nearest-design lookup — and its LOM image is accurate to 0.70% in g and 1.71% in χ, versus 4.03%/28.7% for lookup. Out of hull, *every* data-driven method degrades to ∼3.5–4% capacitance error and $\gtrsim$20% χ error (Figure **??**): extrapolation is not a model-selection problem, it is a regime change — the empirical motivation for Tier 2.

4.2 Routed solver benchmarks

Figure **??** reports mean wall-clock per verified design solve (two excitations, shared context) across grid sizes $n \in \{255, 511, 1023, 2047\}$ on held-out test designs, for every fixed pipeline and for the router. Figure **??** shows what the router learned; Figure **??** shows per-instance ratios to the oracle.

TODO-NUMBERS: router-vs-oracle ratio, speedups vs each fixed pipeline, win rates, worst-case per-instance ratio, decision overhead, OOD split results, verification-failure count (expected 0), escalation statistics.

4.3 Discussion

The benchmark's central pattern is a crossover: sparse direct dominates small grids; surrogate-initialized matrix-free pipelines dominate large grids by skipping assembly and setup entirely; AMG-CG is the strong middle. A fixed choice pays the crossover penalty somewhere; the router pays a microsecond decision everywhere. HINTS' fixed cadence — the same corrector network, the

same smoothers — is dominated at every size measured, confirming that *when* to spend surrogate inference matters more than *whether*.

5 Related work

Hybrid neural–classical solvers. HINTS [2] demonstrated that a DeepONet corrector interleaved with relaxation damps complementary spectral modes; its cadence, however, is a fixed hyperparameter, and our measurements show the schedule is dominated by plain classical pipelines on this workload at every resolution. The greedy PDE router [3] learns *which* operator to apply per iteration with approximation guarantees under contractive error propagation; we inherit its central premise — no single method dominates — but move the decision to the instance level, target verified wall-clock, and add the escalation guardrail, trading per-iteration optimality theory for deployability. **Design databases.** SQuADDS [1] provides the validated data and a physics-based scaling interpolation; its authors report percent-level accuracy in distribution and degradation outside the pre-simulated hull, consistent with what we measure for all data-driven predictors, including ours. **Classical practice.** pyamg's smoothed-aggregation CG [5] is the strong open-source baseline for these SPD systems, and sparse direct factorization is unbeatable at small scale — both are therefore pipelines inside the router rather than external competitors.

6 Limitations and roadmap

v0's field solver is 2D plan-view electrostatics: the right benchmark for routing logic and for surrogate-initialized iteration, but not a replacement for 3D extraction; Tier 1 carries the 3D realism today by training directly on Q3D matrices. The near-term roadmap is (i) a 3D backend via AWS Palace with the same router and verification contract, (ii) eigenmode analyses (resonator frequency and κ) where the SQuADDS cavity-claw tables provide validated anchors, (iii) cost-model transfer across hardware via online refitting on engine logs, and (iv) uncertainty-aware Tier-1/Tier-2 handoff (hull-distance triggers) so the router also decides *whether to solve at all*. The engine logs every measurement needed to retrain both the cost model and the surrogate, so each customer deployment improves its own routing.

References

[1] S. Shanto et al. SQuADDS: A validated design database and simulation workflow for superconducting qubit design. *Quantum* 8, 1465 (2024).

[2] E. Zhang et al. Blending neural operators and relaxation methods in PDE numerical solvers. *Nature Machine Intelligence* (2024).

[3] S. Rayan, Y. Patel, A. Tewari. A greedy PDE router for blending neural operators and classical methods. arXiv:2509.24814 (2025).

[4] J. Koch et al. Charge-insensitive qubit design derived from the Cooper pair box. *Phys. Rev. A* 76, 042319 (2007).

[5] N. Bell et al. PyAMG: Algebraic multigrid solvers in Python. *JOSS* 8(87), 5495 (2023).

[6] Z. Minev et al. Circuit quantum electrodynamics (cQED) with modular quasi-lumped models. arXiv:2103.10344 (2021).